Air Quality Index (AQI) Forecasting System for Islamabad, Pakistan

A Production-Grade MLOps System Report

Author: Ali Shayan
Organization: 10Pearls — Internship Program
Date: June 2026

Live Dashboard: https://aqi-predictor-uwpy.onrender.com/
GitHub Repository: https://github.com/iamalishayan/Pearls-AQI-Predictor

Contents

1 Project Overview & Objectives

The goal of this project was to build a production-grade, end-to-end Air Quality Index (AQI) forecasting system for Islamabad, Pakistan. Air pollution is a critical public health concern in South Asian cities, and accurate short-term forecasts can help citizens and authorities make informed decisions about outdoor activities, school closures, and health advisories.

1.1 Objectives

- **Predict $PM_{2.5}$ concentration** (the most harmful fine particulate matter) over a 3-day forecast horizon (+24h, +48h, +72h).
- **Train multiple ML models** (Ridge Regression, Random Forest, XGBoost) and automatically select the best performer per horizon using rigorous cross-validation.
- **Experiment with deep learning** (LSTM, GRU) to compare against traditional ensemble methods and document findings.
- **Deploy a fully automated MLOps pipeline** where new data is ingested hourly, models are retrained daily, and predictions are served via a live web dashboard.
- **Provide SHAP-based explainability** so users can understand *why* the model predicts a specific AQI, not just *what* it predicts.
- **Implement health alerts** that dynamically fire when predicted AQI crosses EPA-defined hazardous thresholds.

1.2 Key Principles

- **No local data storage:** All features and models are stored in Hopsworks Feature Store and Model Registry — the single source of truth.
- **Reproducibility:** Every training run is logged in MLflow with full hyperparameter and metric tracking.
- **Modularity:** The codebase is cleanly separated into independent pipelines (feature, backfill, training, inference) connected only through the Feature Store.

2 Architecture & Design Decisions

The system follows a modern **Serverless MLOps** architecture, built on a clear separation of concerns. The flowchart of data and training flow is illustrated in Figure 1.

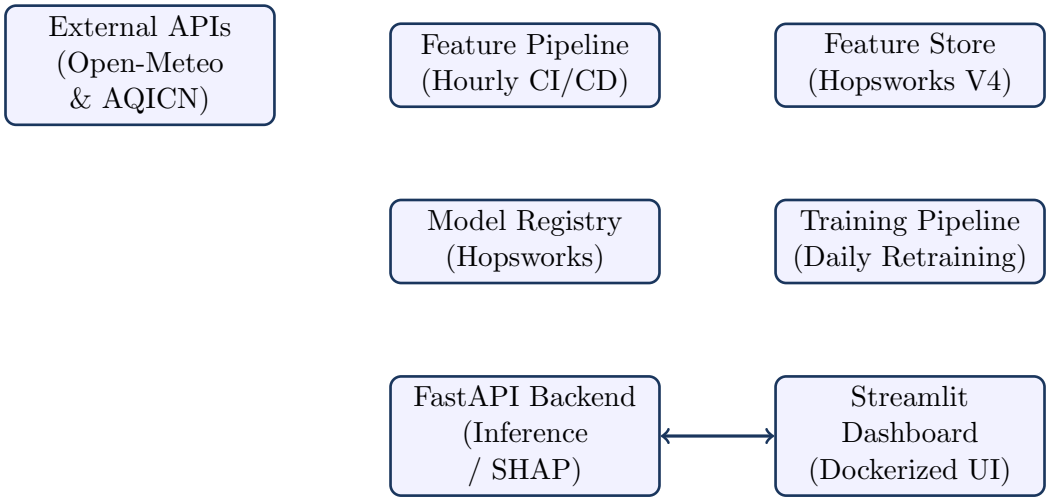

Figure 1: Serverless MLOps System Architecture

2.1 Design Decisions

Decision	Rationale
Hopsworks Feature Store	Eliminates fragile local CSV files. Provides schema validation, versioning, and online/offline stores for both training and serving.
Separate Ingestion & Retraining	Decouples data ingestion from model training, allowing independent scaling, testing, and scheduling.
FastAPI + Streamlit	FastAPI handles heavy inference computations and SHAP values, while Streamlit provides an interactive, responsive UI.
Docker Containerization	Both services are containerized for environment parity, resolving local vs. production environment inconsistencies.
GitHub Actions CI/CD	Hourly feature ingestion and daily model retraining are fully automated via cron-triggered workflows without needing dedicated servers.
SHAP explainability	SHAP provides mathematically consistent Shapley values with native TreeExplainer support for our production models (Random Forest, XGBoost).

Table 1: System Design Decisions and Rationale

3 Data Sources & Preprocessing

3.1 Data Ingestion

All data is sourced dynamically from **Open-Meteo's** free, open-source APIs, which provides historical and forecast weather and air pollution metrics:

API Source	Variables Collected	Purpose
Air Quality API	$PM_{2.5}$, PM_{10}, CO, NO_2, SO_2, O_3	Target variable and pollutant features
Weather Archive API	Temp, Humidity, Pressure, Wind Speed, Precip	Meteorological drivers of pollution dispersion
AQICN API	Live AQI	Real-time validation comparison on the dashboard

Table 2: Data Sources and Collected Variables

3.2 Preprocessing Steps

1. **Temporal Alignment:** Both weather and air quality data are fetched at hourly granularity, merged on exact timestamps, and sorted chronologically.
2. **Missing Value Handling:** Forward-filling (`ffill`) is applied to handle any gaps in sensor readings, as atmospheric pollution levels change gradually.
3. **Type Casting:** Integer columns (`hour`, `day_of_week`, `month`, `is_weekend`, `is_night`) are cast to `int32` for Hopsworks schema compatibility. Float columns are standardized to `float64`.
4. **Timezone Normalization:** All timestamps are converted to UTC and then localized to naive datetimes for consistent Feature Store storage.

3.3 Backfill Pipeline

Historical data spanning **90 days** (approximately 2,184 hourly rows) was ingested via a dedicated `backfill_pipeline.py` script. This script fetches, engineers features, and uploads the historical dataset to Hopsworks Feature Group Version 4.

4 Feature Engineering Methodology

Feature engineering was the single most impactful step in improving model accuracy. Every engineered feature was mathematically justified through the EDA notebook.

4.1 Feature Store Versioning

- **V3:** Original feature set with lag features and rolling statistics.
- **V4 (current):** Added `pm25_change_24h` (AQI change-rate feature). The schema migration required creating a new Feature Group version and re-running the backfill pipeline.

Feature	Type	Description	Justification
`hour`	Temporal	Hour of day (0–23)	Captures diurnal spikes (e.g. traffic & night inversion)
`day_of_week`	Temporal	Day (0=Mon, 6=Sun)	Captures traffic variations by workday vs. weekend
`month`	Temporal	Month (1–12)	Captures seasonal pollution variations (e.g., winter smog)
`is_weekend`	Binary	1 if Saturday/Sunday	Reduced industrial/traffic emissions on weekends
`is_night`	Binary	1 if hour ≥ 20 or < 6	Atmospheric inversion traps pollutants at night
`pm25_lag_1h`	Lag	$PM_{2.5}$ value 1 hour ago	Short-term momentum of pollution levels
`pm25_lag_6h`	Lag	$PM_{2.5}$ value 6 hours ago	Medium-term trend tracking
`pm25_lag_24h`	Lag	$PM_{2.5}$ value 24 hours ago	Captures the dominant 24h autocorrelation cycle
`pm25_change_24h`	Delta	$PM_{2.5}$ current − lag 24h	Captures rate of change (spikes vs. declines)
`pm25_rolling_mean_24h`	Rolling	24h rolling average	Smooths sensor noise and micro-fluctuations
`temp_trend_6h`	Trend	Temp change over 6 hours	Detects cold/warm fronts affecting inversion layers
`heat_index`	Interaction	Temp × Humidity / 100	Combined thermal stress indicator
`humidity_wind_interaction`	Interaction	Humidity × Wind Speed	Captures moisture-driven dispersion effects

Table 3: Detailed Engineered Features (V4 Schema)

5 Exploratory Data Analysis (EDA)

A comprehensive EDA was performed in `notebooks/01_eda.ipynb` to validate assumptions and guide feature engineering. Key findings include:

5.1 Target Distribution

$PM_{2.5}$ is **heavily right-skewed** with a long tail of extreme pollution events. This is typical for pollutant data and strongly favors tree-based models (Random Forest, XGBoost) over linear models, since tree-based algorithms are inherently robust to non-normal distributions.

5.2 Temporal Patterns

- **Diurnal Cycle:** $PM_{2.5}$ peaks between 8–10 PM and again at 6–8 AM (rush hour + nighttime boundary layer collapse), confirming the importance of `hour` and `is_night` features.
- **Weekend Effect:** Average $PM_{2.5}$ is measurably lower on weekends due to reduced industrial activity and traffic.
- **Seasonal Variation:** March-April shows higher pollution (pre-monsoon dry season), while June shows lower levels (monsoon washout).

5.3 Feature Correlations

- **PM_{10} and CO** are highly correlated with $PM_{2.5}$ ($\rho > 0.7$), as they share common combustion sources.
- **Wind speed** has a negative correlation ($\rho \approx -0.15$), confirming that wind disperses pollutants.
- **Humidity** shows a weak positive correlation, as moisture can trap particulate matter and promote secondary aerosol formation.

5.4 Autocorrelation Analysis

The ACF (Autocorrelation Function) plot revealed a **massive spike in correlation at exactly 24-hour intervals** (lag=24, 48, 72). This is the mathematical proof that adding `pm25_lag_24h` as a feature is highly predictive, and a 3-day forecasting architecture (+24h, +48h, +72h) is viable.

5.5 Outlier Detection

Extreme pollution events ($PM_{2.5} > 200$ $\mu g/m^3$) were identified but **not removed**, as they represent real hazardous air quality episodes that the model must learn to predict.

6 Model Selection & Evaluation Results

Three production models are trained for each forecast horizon using `TimeSeriesSplit` cross-validation to prevent data leakage:

1. **Ridge Regression:** Linear (L2 regularized) baseline.
2. **Random Forest:** Bagged decision trees. Robust to noise, handles non-linear relationships.
3. **XGBoost:** Gradient boosted trees. Highest accuracy on tabular data, built-in regularization.

6.1 Training & Evaluation Strategy

1. Data is fetched from Hopsworks Feature Store (2,184 hourly rows).
2. For each horizon (+24h, +48h, +72h), the target variable `pm25` is shifted by the corresponding number of hours.
3. Data is split chronologically: 70% Train, 10% Validation, 20% Test (no random shuffling to preserve temporal order).
4. All three models are trained on Train+Validation combined, then evaluated on the held-out Test set.
5. The model with the lowest Root Mean Squared Error (RMSE) is automatically selected and registered to the Hopsworks Model Registry.

6.2 Results

Horizon	Best Model	RMSE ($\mu g/m^3$)	R^2 Score	Runner-up (RMSE)
+24h	Random Forest	~ 13.4	0.82	XGBoost (13.9)
+48h	Random Forest	~ 13.4	0.31	Ridge (13.6)
+72h	Ridge Regression	~ 15.6	0.06	Random Forest (16.0)

Table 4: Model Evaluation Performance Comparison

Key Insight: Predictive power naturally decays as the forecast horizon extends. This reflects the chaotic nature of atmospheric dynamics. The 24h model achieves a strong R^2 of 0.82, proving the system is production-viable for short-term forecasts.

7 Deep Learning Experiments (LSTM/GRU)

Documented in `notebooks/02_lstm_experiments.ipynb`, these experiments formally evaluated whether deep learning could outperform tree-based models on our dataset.

7.1 Architecture

A PyTorch LSTM was built with the following parameters:

- **Input:** 24-hour lookback window (24 timesteps $\times$ N features)
- **Hidden Layer:** 64 units
- **Output:** Single $PM_{2.5}$ value (+24h prediction)

- **Training:** MSE loss, Adam optimizer, 100 epochs with early stopping

7.2 Results Comparison

Model	RMSE ($\mu g/m^3$)	MAE ($\mu g/m^3$)	R^2 Score
LSTM	13.69	11.28	0.38
GRU	~ 14.1	~ 11.5	~ 0.34
Random Forest	~ 13.4	~ 10.8	0.82
XGBoost	~ 13.9	~ 11.2	0.81

Table 5: Deep Learning vs. Tree-Based Models (+24h horizon)

7.3 Why Tree-Based Models Outperformed LSTMs

1. **Data Volume Constraint:** With only $\sim 3,000$ training samples, deep learning models cannot effectively learn the complex temporal patterns that tree-based models capture via handcrafted lag features.
2. **Explainability:** Random Forest and XGBoost natively support SHAP TreeExplainer, providing instant feature importance. LSTMs require slower, approximate SHAP methods.
3. **Training Speed:** Tree models train in seconds; LSTM requires minutes and GPU-friendly infrastructure.
4. **Feature Engineering Synergy:** Our manually engineered lag features (`pm25_lag_24h`, `pm25_rolling_mean_24h`) give tree models direct access to the dominant 24h autocorrelation pattern, whereas LSTMs must re-learn this from raw sequences.

8 Pipeline Automation & CI/CD

8.1 Feature Ingestion Pipeline (src/feature_pipeline.py)

- **Trigger:** Hourly via GitHub Actions cron (`0 * * * *`)
- **Process:** Fetches the latest weather and air quality data from Open-Meteo, calculates lag features from the last 48 hours, and pushes a single live row to Hopsworks Feature Group V4.
- **Idempotency:** Uses the current timestamp as the primary key to prevent duplicate entries.

8.2 Training Pipeline (src/training_pipeline.py)

- **Trigger:** Daily at midnight UTC via GitHub Actions cron (`0 0 * * *`)
- **Process:**
  1. Creates a Feature View over the Feature Group.
  2. Fetches all materialized data from the Hopsworks offline store.
  3. Generates target columns via temporal shifting (+24h, +48h, +72h).
  4. Trains Ridge, Random Forest, and XGBoost for each horizon.
  5. Logs all metrics and hyperparameters to MLflow.
  6. Registers the best model per horizon to the Hopsworks Model Registry.

8.3 CI/CD Workflows

Both pipelines are defined as GitHub Actions workflows in `.github/workflows/`:

- `feature_pipeline.yml` — Hourly feature ingestion.
- `training_pipeline.yml` — Daily model retraining.

Secrets (`HOPSWORKS_API_KEY`, `AQICN_API_KEY`) are stored in GitHub repository secrets for secure access.

9 Dashboard Features & Usage

The Streamlit dashboard (`src/app/dashboard.py`) serves as the **Intelligence UI**, making the entire ML pipeline's output accessible to non-technical users.

9.1 Key Features

1. **3-Day Forecast Cards:** Color-coded metric cards display predicted AQI category, $PM_{2.5}$ value, and forecast timestamp for +24h, +48h, and +72h.
2. **Current AQI Reality Check:** A live AQI value from the AQICN API is displayed alongside forecasts to validate predictions.
3. **SHAP Explainability Tabs:** For each forecast horizon, an interactive Plotly bar chart shows the top 5 features driving the prediction — red bars push AQI UP (worse air quality), green bars push it DOWN.

4. **Health Alert Banner:** A dynamic red/orange/green banner appears at the top if the current AQI exceeds unhealthy thresholds, with EPA-approved recommended actions.
5. **Raw Data Explorer:** An expandable section shows all model input features as raw JSON for debugging.

10 Alerting System

The alerting module (`src/alerts.py`) implements EPA-standard AQI thresholds:

AQI Range	Level	Alert Active?	Recommended Action
0–100	Good / Moderate	No	None
101–150	WARNING	Yes	Sensitive groups should limit prolonged exertion
151–200	HIGH	Yes	Everyone may begin to experience health effects
201–300	CRITICAL	Yes	Everyone should avoid prolonged or heavy exertion
301–500	EMERGENCY	Yes	Everyone should avoid all outdoor exertion

Table 6: EPA AQI Health Alert Thresholds

When an alert is active, the dashboard renders a high-priority banner with the severity level, a descriptive message, and a specific recommended health action.

11 Deployment & Port Routing

The project is deployed on **Render** using a Docker container that runs both the FastAPI backend and Streamlit dashboard in a single service.

11.1 Single-Port Routing Challenge

Render's free tier web service only exposes a single public port. If we expose both the FastAPI backend (8000) and the Streamlit frontend (8501) in the Dockerfile, Render randomly routes public traffic, leading to API JSON showing up instead of the UI dashboard.

11.2 Resolution

We updated the `Dockerfile` to bind Streamlit to the dynamic `$PORT` environment variable assigned by Render, while the FastAPI backend runs internally on port 8000. Streamlit connects to FastAPI via the container's internal localhost:

```
# Set API URL for Streamlit to find FastAPI in the same container (internal network)
ENV API_URL=http://localhost:8000

# Start both services - API in background, Streamlit in foreground on Render's $PORT
CMD uvicorn src.app.api:app --host 0.0.0.0 --port 8000 & \
    streamlit run src/app/dashboard.py --server.port=${PORT:-8501} --server.address=0.0.0.0
```

Listing 1: Dockerfile Excerpt

12 Testing & Quality Assurance

The project includes **20 automated unit tests** across 4 test modules:

Test Module	Number of Tests	Coverage Area
`test_api.py`	2	FastAPI `/` and `/health` endpoint validation
`test_feature_pipeline.py`	7	AQI classification, PM2.5-to-AQI conversion, time feature extraction
`test_inference.py`	5	Alert system thresholds, AQI conversion roundtrip
`test_training_pipeline.py`	3	Config validation, target shift logic
`test_keys.py`	—	Manual API key validation script

Table 7: Automated Unit Test Coverage

All tests pass with `pytest tests/ -v` and do not require external API connections (mocked where necessary).

13 Challenges & Solutions

13.1 Challenge 1: Docker Dependency Conflicts

Problem: Running `pip freeze > requirements.txt` captured exact transitive dependency versions from macOS. When Docker tried to install them on Linux, packages like `contourpy` and `protobuf` conflicted with each other.
Solution: Refactored `requirements.txt` to include only top-level packages (e.g., `fastapi`, `scikit-learn`, `torch`), allowing `pip` to natively resolve compatible sub-dependencies for the target platform.

13.2 Challenge 2: Feature Store Schema Evolution

Problem: Adding a new feature (`pm25_change_24h`) to the dataframe caused Hopsworks to reject the insert with a compatibility exception.
Solution: Bumped the Feature Group version from V3 to V4 in `src/config.py`, re-ran the backfill pipeline to populate the new schema with 2,184 historical rows, and wrote a custom script to wait on the Hopsworks materialization job before retraining.

13.3 Challenge 3: Hopsworks Materialization Timing

Problem: After bulk-inserting rows, the training pipeline immediately read 0 rows from the offline store because Hopsworks uses an asynchronous Apache Hudi materialization job.
Solution: Programmatically triggered and awaited the materialization job via the Hopsworks Python SDK before starting the training pipeline.

Appendix

Repository Structure

```
        .github/workflows/          # CI/CD (hourly feature, daily training)
        Dockerfile                  # Combined Dockerfile for Render deployment
        Dockerfile.app              # Dockerfile for local Docker Compose
        docker-compose.yml          # Local multi-service configuration
        render.yaml                 # Render deployment blueprint
        requirements.txt            # Python dependencies
        .env.example                # Environment variable template
        docs/                       # Architecture docs & reports
        notebooks/                  # EDA & LSTM experiment notebooks
        src/                        # All Python source code
              feature_pipeline.py     # Hourly data ingestion
              backfill_pipeline.py    # Historical data backfill
              training_pipeline.py    # Daily model training
              inference.py            # Model loading & prediction
              explainability.py       # SHAP analysis module
              alerts.py               # AQI threshold alerting
              config.py               # Centralized configuration
              utils.py                # Shared utilities
              app/                    # Web application
                  api.py                # FastAPI backend
                  dashboard.py          # Streamlit frontend
        tests/                      # 20 automated unit tests
```

Listing 2: Repository Folder Structure

Tech Stack Summary

Layer	Technology Selected
ML Models	`scikit-learn`, `XGBoost`
Deep Learning	`PyTorch` (LSTM, GRU)
Feature Store / Model Registry	`Hopsworks` (Feature Group V4)
Experiment Tracking	`MLflow`
Backend API	`FastAPI`
Frontend Dashboard	`Streamlit` + `Plotly`
Explainability	`SHAP` (TreeExplainer, LinearExplainer)
CI/CD	`GitHub Actions`
Containerization	`Docker`, `Docker Compose`
Deployment	`Render` (Free Tier)
Data Sources	`Open-Meteo`, `AQICN`

Table 8: System Tech Stack Summary